



**UNIVERSITY
OF WARWICK**

CCSE CW1

Starfleet Academy Operations Inventory Management System Report

Software Security Coursework

Student: Syed Imad Uddin

August 10, 2026

Contents

1	Implementation phase	2
1.1	Requirements	2
1.1.1	Requirements Mapping	3
1.2	System Overview	3
1.2.1	Architecture	3
1.2.2	Code stack and why	5
1.2.3	The Models	6
1.2.4	The Routes	7
1.3	Code walkthrough	8
1.3.1	The Login/Role Required Decorators	8
1.4	Test cases	10
2	Critical Reflection	11
2.1	How secure build practices reinforce the integrity and trustworthiness	11
2.2	How does this system align with SSCoP	12
2.3	How does the IMS support or fall short of the SSCoP principles . . .	13

1 Implementation phase

1.1 Requirements

The three scenarios below are given as BDD requirements. Each asks for something specific:

1. RBAC needs an identity and a decision point checking it on every relevant action, not just at login.
2. Inventory records need writes to be attributable and explainable later, even once the underlying data is gone.
3. Audit logging, “any user performs an action” rather than “any write action”, is broader than the other two.

Feature: Operations Inventory Management System for Starfleet Academy

As a Starfleet administrator

I want to manage the inventory of scientific resources and archival materials efficiently

So that I can ensure accurate tracking, availability, and secure access across the fleet.

Scenario: Role-Based Access Control (RBAC)

Given different user roles (Command Officer, Cadet)

When a user logs into the system

Then the system should grant access based on their Starfleet clearance level

And restrict unauthorised actions such as modifying inventory or accessing mission audit logs.

Scenario: Define and Manage Inventory Records

Given I am logged in as a Command Officer

When I add, update, or remove a resource from the inventory

Then the system should reflect the changes

And maintain a history of modifications for audit and mission traceability.

Scenario: Audit Logging of System Activity

Given any user performs an action

When the action is completed

Then the system should log the event with stardate, user ID, and action details

So that Starfleet administrators can review system usage and detect anomalies or breaches.

1.1.1 Requirements Mapping

I've mapped each scenario to the implementation and the test that proves it.

Table 1: Requirements to implementation and evidence mapping

Scenario	Implementation	Evidence
Role-Based Access Control (RBAC)	role_required / login_required decorators (app.py) gate every write route and the audit log, checked against the role stored in the signed session cookie set at login	test_cadet_blocked_from_add_item, test_officer_permitted_to_add_item
Define and Manage Inventory Records	add_inventory_item and inventory_item (GET/PUT/POST/DELETE) handle create/update/remove. Removal is a soft delete via is_removed. Every change writes a row through record_history() into InventoryHistory	test_history_recorded_for_create_update_remove
Audit Logging of System Activity	AuditLog model plus before_request / after_request hooks log every request with timestamp, user ID, and outcome (success/denied/unauthenticated/failure)	test_audit_log_coverage_unauthenticated_denied_and_success

1.2 System Overview

1.2.1 Architecture

I built it so every request passes through the decorators and the before_request / after_request hooks before reaching a view function. This makes the audit log unconditional, and is why RBAC cannot be bypassed by hitting a route directly instead of going through the browser.

The diagram below describes the request pipeline.

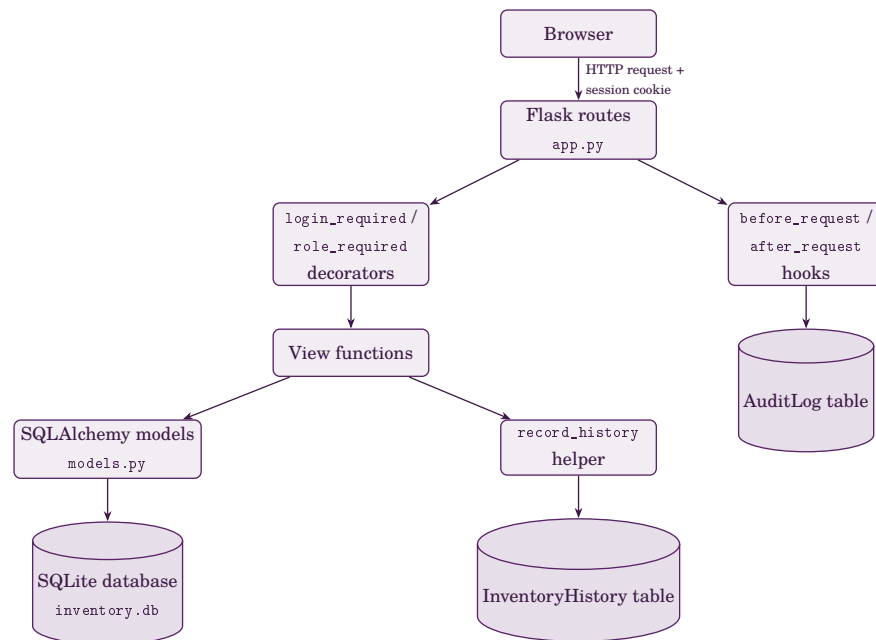


Figure 1: Request pipeline architecture

app.py has two layers on top of this: five helper functions (two decorators for auth, one for writing history rows, two request hooks for audit logging) and eleven route functions. Almost every route follows the same structure:

1. Check the request is allowed
2. Read the submitted data
3. Validate it
4. Write to the database
5. Flash a message and redirect or return JSON

Once that's clear for one route, the rest are the same steps with different fields, not new logic each time. The exceptions are the decorators themselves and /inventory/<id>, which handles four HTTP methods in one function.

Helper functions

Function	Type	Purpose
login_required(f)	Decorator	Redirects to login if no session exists
role_required(*roles)	Decorator factory	Redirects if no session, or if the session's role isn't in roles
record_history(item, action)	Helper	Writes one snapshot row to InventoryHistory

Function	Type	Purpose
reset_audit_outcome()	before_request hook	Resets g.audit_outcome to "success" at the start of every request
log_request(response)	after_request hook	Writes one AuditLog row per request, using whatever g.audit_outcome ended up as

Routes

Function	Route	Access	Purpose
home()	GET /	Public	Login page if logged out, dashboard if logged in
list_inventory()	GET /inventory	Any logged in role	Lists non removed items as JSON
inventory_item(item_id)	GET/PUT/POST/DELETE /inventory/<id>	GET: any role. PUT/POST/DELETE: officer	View, update, or soft delete one item
inventory_item_history(item_id)	GET /inventory/<id>/history	Any logged in role	Returns that item's modification history as JSON
audit_log()	GET /audit-log	Officer only	Renders the audit log page
login()	POST /login	Public	Authenticates and sets the session
logout()	POST /logout	Any logged in role	Clears the session
add_inventory_item()	POST /inventory/add	Officer only	Creates a new item
add_user()	POST /user/add	Officer only	Creates a new user account
temp_add_user()	GET /user/temp_add	Public (development only)	a test cadet or officer account
temp_add_item()	GET /item/temp_add	Public (development only)	a test inventory item

1.2.2 Code stack and why

Python

- Flask

- SQL Alchemy
- SQLite
- HTML + Jinja template engine

I chose Python as it's my strongest language, versatile, and quick to spin up a small web server in, with a large library ecosystem for the API and front end.

I've used SQLAlchemy as the ORM since it's the most common choice in industry, with plenty of documentation available. SQLite avoids the overhead of a live server database, since this project is local and doesn't need concurrent access from multiple users.

I've opted for HTML and Jinja templating as I'm most comfortable in this stack, not what's typically used in industry, but easier to work with for a project this size than a separate front end framework would have been.

I originally built login with JWT (flask-jwt-extended), the standard approach for a REST API. I switched to session cookies once building the browser GUI started, because JWT only works if the client attaches the token to every request itself, meaning JavaScript reading the token and setting a header on every fetch call. Since the GUI needed to run on plain HTML forms with no JavaScript, that was not compatible. A session cookie is sent back by the browser automatically, so switching let me drop the JavaScript layer entirely and keep the front end to just Jinja templates and forms.

1.2.3 *The Models*

The database has four models:

1. User
2. InventoryItems
3. InventoryHistory
4. AuditLog

User stores the username, a hashed password (via werkzeug's `generate_password_hash`, so nothing is stored in plaintext), and a role field: `cadet` or `command_officer`.

InventoryItems stores the name and quantity of a resource, plus an `is_removed` flag. I added `is_removed` before history or audit logging existed as a requirement, since permanently deleting a row would leave nothing for a history table to point back to. Soft deleting just flags the row instead.

InventoryHistory and AuditLog both carry a `user_id` linking back to User, making it possible to say *who* did something, not just that it happened. Rather than store a diff each time an item changes, InventoryHistory stores a full snapshot after each change. This uses more storage than a diff would, but any row can be read back on its own without replaying earlier changes to work out what it looked like.

```
# logs one history row per create/update/remove - snapshot of the item after the change
def record_history(item, action):
    entry = InventoryHistory(
        item_id=item.id,
        user_id=session["user_id"],
        action=action,
        name_snapshot=item.name,
        quantity_snapshot=item.quantity,
    )
    db.session.add(entry)
    db.session.commit()
```

Figure 2: `record_history()`, the helper that writes a snapshot row

Model	Key fields	Purpose
User	id, username, password_hash, role	Accounts and role based access (cadet / command_officer)
InventoryItems	id, name, quantity, is_removed	Inventory resources. is_removed is the soft delete flag
InventoryHistory	id, item_id (FK), user_id (FK), action, timestamp, name_snapshot, quantity_snapshot	One row per create/update/remove, a full snapshot rather than a diff
AuditLog	id, user_id (FK, nullable), path, method, outcome, timestamp	One row per request. user_id is null when the request was unauthenticated

1.2.4 The Routes

I split the routes into reads (any logged in role) and writes (command officer only), so the RBAC boundary sits at that read/write line rather than per route.

`/` shows the login page if nobody's logged in, or the dashboard if they are. `/login` and `/logout` handle authentication through the session cookie. `/inventory` (GET) lists all non removed items to any logged in user, and `/inventory/add` (POST) lets an officer create one.

`/inventory/<id>` is one route accepting GET, PUT, POST and DELETE, with GET open to any role and PUT/DELETE restricted to officers. It is one route rather than four because curl and Postman can send real PUT/DELETE requests, but a plain HTML form can only send GET or POST, so the edit and remove buttons both POST

to the same URL, with a hidden `_action=delete` field telling the route which one it is meant to be doing.

```
# Route: Inventory item - view is any role, edit/remove is officer only
@app.route("/inventory/<int:item_id>", methods=["GET", "PUT", "POST", "DELETE"])
@login_required
def inventory_item(item_id):
    item = InventoryItems.query.filter_by(id=item_id, is_removed=False).first()
    if not item:
        return jsonify({"error": "item not found"}), 404

    if request.method == "GET":
        return jsonify(item.to_dict())

    # everything below here is a write - officer only
    if session.get("role") != "command_officer":
        g.audit_outcome = "denied"
        flash("not authorised for this action", "error")
        return redirect(url_for("home"))

    # browser form can't send DELETE, so it sends _action=delete instead
    if request.method == "DELETE" or request.form.get("_action") == "delete":
        item.is_removed = True
        db.session.commit()
        record_history(item, "remove")
        flash("item removed", "success")
    else:
        name = request.form.get("name")
        quantity = request.form.get("quantity")
        if name:
            item.name = name
        if quantity:
            item.quantity = int(quantity)
        db.session.commit()
        record_history(item, "update")
        flash("item updated", "success")

    if request.method in ("PUT", "DELETE"):
        return jsonify(item.to_dict())
    return redirect(url_for("home"))
```

Figure 3: `inventory_item()`, the route handling all four methods

`/inventory/<id>/history` returns an item's modification history, open to any role. `/user/add` lets an officer create an account. `/user/temp_add` and `/item/temp_add` are unauthenticated development routes for seeding a test user and item, needed because once `/user/add` requires an officer, nothing can create the first one.

`/audit-log` lists every logged action and is restricted to officers, satisfying the requirement to restrict access to mission audit logs.

1.3 Code walkthrough

1.3.1 The Login / Role Required Decorators

A decorator takes a function as input and returns a replacement function in its place. `login_required` and `role_required` are applied to routes such as `add_inventory_item` to decide whether the real route runs.

The decorator runs once, at startup, when Python reads through `app.py`. It can't check a session on every visit since no request exists yet at that point, the app hasn't started serving anything. So `login_required` builds a second function, `wrapper`, and returns that instead. `wrapper` is what Flask actually calls on every request, checking session state fresh each time.

If `user_id` isn't in the session, wrapper redirects to login and the real route never runs. Otherwise the real route runs as normal.

```
# redirects to login if no session
def login_required(f):
    @wraps(f)
    def wrapper(*args, **kwargs):
        if "user_id" not in session:
            g.audit_outcome = "unauthenticated"
            flash("login required", "error")
            return redirect(url_for("home"))
        return f(*args, **kwargs)
    return wrapper
```

Figure 4: `login_required`

`role_required` does the same check but needs to know which role, and that differs per route: `command_officer` for adding an item, for example. It takes one extra step: `role_required("command_officer")` builds a wrapper that remembers that role, then checks `session.get("role")` against it on every request.

```
# restricts route to given roles, checks login too so decorator order can't mess it up
def role_required(*roles):
    def decorator(f):
        @wraps(f)
        def wrapper(*args, **kwargs):
            if "user_id" not in session:
                g.audit_outcome = "unauthenticated"
                flash("login required", "error")
                return redirect(url_for("home"))
            if session.get("role") not in roles:
                g.audit_outcome = "denied"
                flash("not authorised for this action", "error")
                return redirect(url_for("home"))
            return f(*args, **kwargs)
        return wrapper
    return decorator
```

Figure 5: `role_required`, the decorator factory

I used `@wraps(f)` from Python's `functools` on both decorators. Without it, every wrapper function is literally named `wrapper`, and Flask uses that name internally to tell routes apart. Two routes sharing the same decorator would look identical to Flask and crash the app on startup. `@wraps(f)` copies the real route's name back onto `wrapper`.

Both decorators only ever trust the session, never anything the request itself claims. The session is a cookie signed by the server, so the browser can send it back but cannot edit it without the server noticing, unlike a form field or header, which is simply whatever the client typed in. The role checked always comes from that signed cookie, set at login, never from anything the client could fill in themselves.

1.4 Test cases

The 4 following areas were outlined in the user requirements and therefore have corresponding tests: login works, roles are enforced both ways, changes are tracked, and actions are logged including refusals. Real automated proof for each, rather than a manual click through I did once and did not repeat.

Test	Proves	Requirement
test_login_success / test_login_failure	Accepts valid creds, rejects invalid	Auth precondition underlying RBAC
test_cadet_blocked_from_add_ item / test_officer_permitted _to_add_item	Roles enforced both ways, not just logged in = allowed	RBAC scenario
test_history_recorded_for_cr eate_update_remove	Create, update, remove lineage captured with correct snapshots each step	Inventory records / modification history scenario
test_audit_log_covers_unauth enticated_denied_and_success	Unauthenticated, denied, and success all logged, not just wins	Audit logging scenario

2 Critical Reflection

2.1 How secure build practices reinforce the integrity and trustworthiness

I compiled and ran every stage against a live version of the app before considering it finished, instead of assuming it worked.

The audit log shows why rechecking implementation against the original requirement matters, not just against my own plan. I had initially scoped it to only `/login` and `/inventory*`, but the user requirement says “*any* user performs *an* action”, broader than what I had built. Separately, the RBAC scenario’s line about restricting access to “mission audit logs” implied a route to view the log needed to exist and be protected, which it did not yet. Neither gap would have been caught by testing what already existed, only by rereading the requirement itself.

Two real bugs came from writing the pytest suite, not manual testing:

- `flask.g` should reset every request, but my test fixture wrapping requests in a single app context let a value leak from one request into the next, meaning a denied action could log as a success, exactly what an audit log exists to prevent. I only found it because a test asserted the exact outcome string, not because the request did not error.
- My tests were quietly writing to the real database, because Flask-SQLAlchemy fixes the database URI in place the moment `db.init_app()` runs rather than lazily on first use, confirmed with a standalone reproduction script before trusting the fix.

Manual testing had not caught either bug.

```
# resets g each request - otherwise it can carry a stale value over from the last one
@app.before_request
def reset_audit_outcome():
    g.audit_outcome = "success"

# logs every request, denied/unauthenticated included - skips /static, not an "action"
@app.after_request
def log_request(response):
    if not request.path.startswith("/static"):
        entry = AuditLog(
            user_id=session.get("user_id"),
            path=request.path,
            method=request.method,
            outcome=g.audit_outcome,
        )
        db.session.add(entry)
        db.session.commit()
    return response
```

Figure 6: The before_request/after_request pair behind the first bug: g.audit_outcome reset on every request, then read back when logging it

Two weaknesses are worth stating plainly rather than leaving for a marker to find:

- SECRET_KEY is hardcoded in app.py rather than read from an environment variable. This is acceptable for a project that never leaves my machine, but it would let anyone with source code access forge session cookies in a real deployment.
- Sessions cannot be revoked early: a compromised account or a changed role would not take effect until that session expired or the user logged out.

2.2 How does this system align with SSCoP

The Software Security Code of Practice (DSIT/NCSC, May 2025) targets commercial vendors selling to business customers, therefore not every principle translates onto a small internal academy system. I have mapped where the fit is an analogy rather than a literal match.

Table 6: Mapping of security requirements and features to SSCoP principles

Security Requirement and Feature	Description	Mapped Principles
Role based access control (role_required)	Server side, deny by default enforcement of Cadet vs Officer on every write route and the audit log	1.4: secure by design/default
Session based authentication	Signed cookie via Flask's built in session mechanism, not a custom rolled scheme	1.1: established secure development framework

Security Requirement and Feature	Description	Mapped Principles
Soft delete (<code>is_removed</code>)	Decided upfront, before history was even a requirement, that removal should not destroy data	1.4: secure by design
Modification history	Snapshot row per create/update/remove: who, when, what	2.2: analogy, not literal. 2.2 is about the <i>build</i> environment, no build pipeline here, but same idea: traceable, attributable changes, just at the application layer
Audit logging (before_request / after_request)	Every request logged with outcome, denials and unauthenticated attempts included	2.2: same analogy, applied to every request not just data changes
pytest suite	6 tests gating each stage before it was considered complete	1.3: clear process for testing before distribution, the most direct match in this table

2.3 How does the IMS support or fall short of the SSCoP principles

I have gone by theme rather than principle since several principles within a theme land on the same assessment.

SSCoP theme	Assessment
1. Secure design and development	Strength: deny by default access control, tested every stage. Gap: no named or documented secure development framework, process was iterative and informal, not structured against a formal threat model.
2. Build environment security	Gap, mostly N/A: no real build or release pipeline, locally run app not a compiled product. Audit logging is the closest analogue but at the wrong layer.
3. Secure deployment and maintenance	Gap: no vulnerability disclosure process, no patching or update mechanism, no dependency risk review of <code>requirements.txt</code> . Assumes a distributed product with an ongoing lifecycle, this project does not really have one.
4. Communication with customers	N/A: no external customers, internal academy system. Stating plainly rather than forcing a mapping onto a theme that does not transfer.

Appendix

A.1 AI Tools

Helped me research SScOP and understand it better.

Struggled with wrappers and decorators, explained them through analogies.

Formatted the report into latex.

Helped debug and write test cases using pytest.

Converted my iPad drawing into a Mermaid diagram.

Helped write the README file.

Helped write HTML code.

Used for grammar, punctuation, and cutting the report down on words.

A.2 Codebase Access Information

See README.md in the project root for setup instructions.

Github Link: https://github.com/Syed-Imad/CCSE_FFA.git